Kotlin e Kotlin Multiplatform

DIM0524 — Sistemas para Dispositivos Móveis · Aula 02 (Sprint 0)

Prof. Fernando · UFRN/DIMAp · 2026.2

> Hoje vocês escrevem e executam código Kotlin diretamente no navegador, sem instalação. A configuração do ambiente local fica como tarefa, com apoio.

Roteiro

Parte	O quê
0	**O combinado** — plano de curso, calendário e avaliação
1	**A linguagem Kotlin** — do zero, com exercícios no navegador
2	**Kotlin e Java 21** — o que é igual, o que muda
3	**Kotlin Multiplatform** — o que é, e por quê

A Parte 0 recapitula o que já foi dito na aula 01, e fica registrada aqui para consulta.

Onde estudar depois

Kotlin Tour, guia oficial da JetBrains — é a ordem que vamos seguir hoje:

> `kotlinlang.org/docs/kotlin-tour-welcome.html`

Cada capítulo tem editor embutido na própria página. Roda no navegador.

Guia	Foco
Kotlin Tour	A linguagem
Android Basics with Compose (Google)	Android e interface
Kotlin Multiplatform Docs	Multiplataforma

Parte 0

0 combinado

Plano de curso, calendário e avaliação. Já vimos na aula 01 — fica aqui para referência.

O material da disciplina

Todo o material está público no GitHub.

> `github.com/fmarquesfilho/sistemas-moveis-2026-2`

Documento	Conteúdo
`docs/CRONOGRAMA.md`	Aula por aula, com as datas efetivas
`docs/AVALIACAO.md`	Sistemática de avaliação
`docs/RUBRICAS.md`	Critérios de cada entrega
`docs/STACK.md`	Tecnologias, bibliotecas e ambiente
`docs/SPRINT-0.md`	Guia da entrega, com templates

Mudamos de tecnologia

Na primeira aula o curso seria em Flutter. Depois da conversa com vocês e de uma revisão do material, o curso passa a ser em **Kotlin Multiplatform**.

O que pesou	Por quê
Transferência	Kotlin é a linguagem nativa de Android. Serve fora deste curso
Maturidade	Compose Multiplatform para iOS é estável desde 2025
Ferramental	Funciona em Windows e Linux, sem Mac para a maior parte

O panorama de plataformas que vimos na aula 01 continua valendo. O código começa hoje.

Avaliação

Unidade	Composição	Fecha em
U1	Sprint 0 (20%) + Sprint 1 (40%) + Sprint 2 (40%)	23/10
U2	Sprint 3 (60%) + Prova (40%)	30/11
U3	Entrega final (60%) + Prova (40%)	11/12

Cada sprint: **50%** entrega técnica · **30%** atividade no repositório · **20%** comunicação

A U1 não tem prova e fecha em 23/10, então vocês conhecem o próprio desempenho antes de decidir sobre a segunda prova.

Duas provas, vale a maior

Data	Prova	Conteúdo
21/10	Prova escrita	Sprints 0 a 2
30/11	Prova de reposição, **opcional**	Cumulativa, Sprints 0 a 3

- Individuais, questões fechadas, no Multiprova, em laboratório
- Consulta permitida a **uma folha A4 manuscrita**, frente e verso
- Quem não fizer a segunda fica com a nota da primeira

Essa mudança veio da turma: a prova estava tarde demais para dar chance de recuperação. Agora a primeira nota sai em outubro.

Como o semestre funciona, e as datas

Sprint 0 (estendida para 4 semanas) + **3 sprints** + **bloco final**. Toda sprint tem aulas presenciais, um encontro online, apresentações e entrega na sexta.

Data	Compromisso
11/09	Entrega da Sprint 0
02/10	Sprint 1 — interface e navegação
21/10	Prova escrita
23/10	Sprint 2 — estado e arquitetura
20/11	Sprint 3 — dados, rede e offline
30/11	Prova de reposição, opcional
11/12	Entrega final

Os feriados deslocam o ritmo de cada sprint. Cronograma aula por aula em `docs/CRONOGRAMA.md`.

O que você entrega em 11/09

1. Repositório **público** com projeto KMP que compila nos alvos **Android e desktop**
2. `ktlintCheck` e `detekt` limpos no GitHub Actions
3. `docs/proposta.md` — visão, MVP, backlog e as decisões justificadas
4. Uma tela em Compose, com ao menos **um componente próprio** e estado elevado
5. Vídeo de 5 minutos

Grupos de **1 a 4** integrantes, formados até 11/09.

Guia completo, com templates e exemplos, em `docs/SPRINT-0.md`.

Visão do produto

Antes de escrever código, responda por que o produto existe:

```
Para [usuários-alvo]                Para agentes de campo de secretarias municipais
Que [problema ou necessidade]       Que registram vistorias em papel e redigitam depois
O [nome] é um [categoria]           O Vistoria é um app de registro em campo
Que [benefício principal]           Que captura foto, GPS e formulário mesmo sem sinal
Diferente de [alternativa]          Diferente de planilhas e formulários de papel
Nosso produto [diferencial único]   Sincroniza sozinho quando a conexão volta
```

Se a equipe não consegue preencher isso, ainda não tem projeto — tem só uma ideia.

Note o que a última linha faz: nomeia o **diferencial técnico** que vai virar a Sprint 3.

MVP e as três decisões

No MVP	Fora do MVP
Cadastro com foto e GPS	Assinatura digital
Formulário com validação	Geração de PDF
Funcionamento offline com fila	Painel web
Sincronização ao reconectar	Relatórios e gráficos

Declarar o que fica **fora** é uma decisão de engenharia, e é o que protege o prazo.

Decisão da Sprint 0	Opções
Plataforma-alvo	Android ou iOS
Backend	Serviço gratuito, API de Web II, ou local + APIs públicas
Coorte	Apresentar online ou em sala

Todas com **justificativa avaliada**. A interface é Compose para todo mundo; SwiftUI por cima conta para o bônus de multiplataforma.

Parte 1

A linguagem

Do início. Abram `play.kotlinlang.org`, que não exige cadastro nem instalação.

O primeiro programa

```
fun main() {
    println("Olá, turma")
}
```

Elemento	Nota
`fun`	Declara uma função
`main()`	Ponto de entrada
`println`	Já disponível, sem `import`
`;`	Opcional. Por convenção, não se usa

> Comparando com Java: não é necessário envolver `main` numa classe. Em Kotlin, funções podem existir no nível do arquivo.

Variáveis: `val` e `var`

```
val nome = "Ana"      // somente leitura — não pode ser reatribuída
var contador = 0      // mutável
contador += 1

// nome = "Bruno"     ← ERRO de compilação
```

O tipo é **inferido**, mas pode ser escrito:

```
val titulo: String = "MUSI"
var ano: Int = 1967
```

Sugestão prática: comece com `val` e mude para `var` quando precisar reatribuir. Em Kotlin, valores imutáveis são o caso mais comum.

Tipos básicos

Tipo	Exemplo	Nota
`Int`	`42`	32 bits
`Long`	`42L`	64 bits
`Double`	`3.14`	Ponto flutuante padrão
`Boolean`	`true`, `false`	—
`Char`	`'a'`	Aspas simples
`String`	`"texto"`	Aspas duplas

Kotlin não separa tipos primitivos de suas versões objeto. Onde Java tem `int` e `Integer`, Kotlin tem apenas `Int`.

```
val ano = 1967          // Int, inferido
val duracao = 3.5       // Double, inferido
```

Interpolação em strings

```
val artista = "Edu Lobo"
val ano = 1967

println("$artista gravou em $ano")
println("Daqui a 10 anos: ${ano + 10}")
```

Saída:

```
Edu Lobo gravou em 1967
Daqui a 10 anos: 1977
```

- `$nome` para uma variável simples
- `${expressão}` para qualquer expressão

> É uma alternativa à concatenação com `+` e ao `String.format`, que costuma deixar o texto mais legível.

Coleções: `List`

```
val ritmos = listOf("ijexa", "baiao", "maracatu")

println(ritmos[0])          // ijexa
println(ritmos.size)        // 3
println(ritmos.first())     // ijexa
println("baiao" in ritmos)  // true
```

`listOf` cria uma lista somente leitura, sem `add` nem `remove`.

```
val mutavel = mutableListOf("ijexa")
mutavel.add("coco")
mutavel += "xote"
```

Aqui aparece de novo a mesma orientação do val: o padrão é imutável.
Criar uma coleção modificável exige escrever mutableListOf, e sinaliza que aquela coleção vai mudar.

Coleções: `Set` e `Map`

```
val generos = setOf("mpb", "forro", "mpb")   // duplicata some
println(generos.size)                         // 2

val anos = mapOf(
    "Ponteio" to 1967,
    "Asa Branca" to 1947
)

println(anos["Ponteio"])       // 1967
println(anos["Inexistente"])   // null
```

`to` cria um par. É uma função comum da biblioteca padrão, não sintaxe especial.

> Todas têm versão mutável: `mutableSetOf`, `mutableMapOf`.

Operações sobre coleções

```
val anos = listOf(1947, 1967, 1969, 1975, 1994)

anos.filter { it > 1960 }        // [1967, 1969, 1975, 1994]
anos.map { it + 1 }              // [1948, 1968, 1970, 1976, 1995]
anos.any { it < 1950 }           // true
anos.all { it > 1900 }           // true
anos.sum()                       // 9852
anos.sorted()                    // crescente
```

`it` é o nome implícito do único parâmetro da lambda.

```
anos.filter { ano -> ano > 1960 }    // com nome explícito, se preferir
```

São as mesmas ideias da Stream API de Java, disponíveis diretamente na coleção, sem as chamadas a `.stream()` e `.collect()`.

`if` é uma expressão

```
val ano = 1967

// como comando, igual a Java
if (ano < 1960) {
    println("antes dos anos 60")
}

// como EXPRESSÃO — devolve valor
val decada = if (ano < 1960) "anos 50" else "anos 60 ou depois"
```

Kotlin não tem o operador ternário `? :`, porque o próprio `if` cumpre esse papel.

> Estruturas que devolvem valor podem ser atribuídas a um `val`, o que costuma reduzir a quantidade de variáveis mutáveis no código.

`when` — `switch` que virou expressão

```
val ritmo = "baiao"

val regiao = when (ritmo) {
    "baiao", "xote"  -> "Nordeste"
    "maracatu"       -> "Pernambuco"
    "ijexa"          -> "Bahia"
    else             -> "não sei"
}
```

Também funciona **sem argumento**, como cadeia de condições:

```
val faixa = when {
    ano < 1950 -> "histórico"
    ano < 1980 -> "clássico"
    else       -> "moderno"
}
```

Intervalos e repetição

```
for (i in 1..5) print(i)          // 12345
for (i in 1 until 5) print(i)     // 1234   (exclui o 5)
for (i in 5 downTo 1) print(i)    // 54321
for (i in 0..10 step 2) print(i)  // 0246810

for (ritmo in listOf("ijexa", "baiao")) {
    println(ritmo)
}

var n = 3
while (n > 0) { n-- }
```

Kotlin não tem a forma `for (int i = 0; i < n; i++)` de Java. A repetição é sempre sobre um intervalo ou uma coleção.

Funções

```
fun somar(a: Int, b: Int): Int {
    return a + b
}
```

Quando o corpo é uma única expressão, use **corpo de expressão**:

```
fun somar(a: Int, b: Int): Int = a + b

fun somar(a: Int, b: Int) = a + b        // tipo de retorno inferido
```

Função que não devolve nada tem tipo `Unit`, e ele é omitido:

```
fun saudar(nome: String) {
    println("Olá, $nome")
}
```

> Tipos de parâmetro são sempre declarados. Apenas o tipo de retorno pode ser inferido.

Parâmetros nomeados e valores padrão

```
fun formatar(
    titulo: String,
    artista: String,
    maiuscula: Boolean = false      // valor padrão
): String {
    val texto = "$titulo — $artista"
    return if (maiuscula) texto.uppercase() else texto
}

formatar("Ponteio", "Edu Lobo")
formatar("Ponteio", "Edu Lobo", maiuscula = true)
formatar(artista = "Edu Lobo", titulo = "Ponteio")   // ordem livre
```

Esses dois recursos reduzem a necessidade de sobrecargas e tornam mais legíveis as chamadas com muitos argumentos — algo bastante presente em Compose.

Classes

```
class Curador(val nome: String, var ativo: Boolean = true) {

    fun apresentar() = "Curador: $nome"

    fun desativar() {
        ativo = false
    }
}

val c = Curador("Ana")
println(c.nome)          // Ana
println(c.apresentar())  // Curador: Ana
c.desativar()
```

- Sem `new`. Basta `Curador("Ana")`
- Parâmetros do **construtor primário** viram propriedades quando marcados `val` / `var`
- `val` gera só o acessor de leitura; `var` gera leitura e escrita

Herança e interfaces

```kotlin
interface Descritivel {
    fun descrever(): String
}

// Classes são FINAIS por padrão. Para herdar, marque `open`.
open class Obra(val titulo: String) : Descritivel {
    override fun descrever() = titulo
}

class Gravacao(titulo: String, val ano: Int) : Obra(titulo) {
    override fun descrever() = "$titulo ($ano)"
}
```

Java	Kotlin
`extends` e `implements`	`:` para os dois
`@Override` opcional	`override` **obrigatório**
Classes abertas por padrão	Classes **finais** por padrão

Kotlin adota como padrão a recomendação de que herança seja um recurso explicitamente planejado: por isso `open` é necessário.

`data class`

Para classes que só **carregam dados**:

```
data class Caracteristica(val dimensao: String, val valor: String)

val a = Caracteristica("ritmo", "ijexa")
val b = Caracteristica("ritmo", "ijexa")

println(a)                      // Caracteristica(dimensao=ritmo, valor=ijexa)
println(a == b)                 // true — igualdade por VALOR
println(a.copy(valor = "baiao"))  // Caracteristica(dimensao=ritmo, valor=baiao)
```

O compilador gera `equals`, `hashCode`, `toString`, `copy` e `componentN`.

Classes seladas

Uma interface comum é **aberta**: qualquer classe, em qualquer lugar, pode implementá-la, e o compilador não sabe quantas existem.

`sealed` fecha essa lista:

```
sealed interface Filtro {
    data class Tem(val dimensao: String, val valor: String) : Filtro
    data class Ou(val opcoes: List<Filtro>) : Filtro
    data class E(val exigencias: List<Filtro>) : Filtro
}
```

O compilador passa a conhecer **todas** as implementações. Por isso consegue verificar se um `when` sobre `Filtro` tratou cada uma delas.

```
val descricao = when (f) {
    is Filtro.Tem -> "${f.dimensao}=${f.valor}"    // f já é Tem aqui
    is Filtro.Ou  -> "alguma das opções"
    is Filtro.E   -> "todas as exigências"
}
```

> Dentro de cada ramo, `f` já tem o tipo específico, sem conversão explícita. É o mesmo *smart cast* do slide de null safety.

Objetos imutáveis e `copy`

```
val original = Caracteristica("ritmo", "ijexa")
val alterada = original.copy(valor = "baiao")

println(original)   // Caracteristica(dimensao=ritmo, valor=ijexa)   ← intacta
println(alterada)   // Caracteristica(dimensao=ritmo, valor=baiao)
```

Objetos imutáveis não são alterados: cria-se uma cópia com as diferenças desejadas.

> Esse padrão aparece bastante em Compose e na gerência de estado, então vale se familiarizar com ele desde já.

Null safety — o problema

Em Java, qualquer referência pode ser `null`, e o compilador não sinaliza isso:

```
String nome = obterNome();   // pode devolver null
nome.length();               // NullPointerException em produção
```

Em Kotlin, `null` **faz parte do tipo**:

```
var titulo: String = "Ponteio"
// titulo = null              ← ERRO de compilação

var apelido: String? = null   // o ? admite nulo
```

`String` e `String?` são **tipos diferentes**.

> Tony Hoare, que introduziu a referência nula em 1965, costuma se referir a ela como um erro caro. Kotlin trata a questão em tempo de compilação, e não de execução.

Null safety — como lidar

```
val apelido: String? = obterApelido()

// apelido.length          ← não compila

apelido?.length                       // chamada segura → Int?
apelido?.length ?: 0                  // valor padrão   → Int
apelido?.let { println(it.uppercase()) }   // só executa se não for nulo

if (apelido != null) {
    println(apelido.length)           // smart cast: aqui já é String
}
```

Operador	Significa
`?.`	Chame só se não for nulo
`?:`	Se o lado esquerdo for nulo, use o direito
`!!`	Afirma que o valor não é nulo; lança exceção caso seja

O operador `!!` desativa a verificação do compilador, então convém usá-lo apenas quando não houver alternativa.

Funções de extensão

Acrescentam comportamento a um tipo **sem herdar dele**:

```
fun String.entreAspas(): String = "\"$this\""

"ijexa".entreAspas()      // "ijexa" com aspas

"ijexa".primeiraMaiuscula()      // "Ijexa"

fun List<Int>.media(): Double = if (isEmpty()) 0.0 else sum().toDouble() / size

listOf(1967, 1969).media()       // 1968.0
```

Dentro da extensão, `this` é o objeto — e pode ser omitido.

> Boa parte da biblioteca padrão é construída assim. Em Java, o equivalente seria uma classe utilitária com métodos estáticos.

Exercício 1 play.kotlinlang.org

```
data class Obra(val titulo: String, val artista: String, val ano: Int)

fun main() {
    val acervo = listOf(
        Obra("Ponteio", "Edu Lobo", 1967),
        Obra("Asa Branca", "Luiz Gonzaga", 1947),
        Obra("Refazenda", "Gilberto Gil", 1975)
    )

    // 1. Imprima só as obras posteriores a 1960
    // 2. Imprima os títulos em maiúscula
    // 3. Escreva uma função que receba a lista e devolva a obra mais antiga
    // 4. Use `when` para classificar cada obra em "histórico" (<1950),
    //    "clássico" (<1980) ou "moderno"
}
```

Cerca de 10 minutos. Se sobrar tempo, transforme o item 3 numa função de extensão de `List<Obra>`.

Parte 2

Kotlin e Java 21

O que vocês já sabem continua valendo — e o que muda.

O que é igual

Roda na JVM	Mesmo bytecode, mesma máquina virtual
Coletor de lixo	Mesmo GC, mesmo modelo de memória
Orientada a objetos	Classes, herança, interfaces, polimorfismo
Tipagem estática	Verificada na compilação
Bibliotecas	Toda a biblioteca de Java está disponível
Ferramental	Maven, Gradle, JUnit, IntelliJ

Interoperabilidade total: um arquivo `.kt` e um `.java` no mesmo projeto se enxergam.

> Kotlin surgiu na JetBrains a partir de uma necessidade concreta: evoluir uma base de milhões de linhas de Java no IntelliJ sem reescrevê-la. A interoperabilidade foi, desde o início, um requisito de projeto.

O que muda — sintaxe

Java	Kotlin
`String nome = "x";`	`val nome = "x"`
`final`	`val` (e é o padrão)
`new Pessoa()`	`Pessoa()`
`public class`	`class` (público é o padrão)
ponto e vírgula obrigatório	opcional
`x instanceof S s`	`x is S` (com smart cast)
`(String) x`	`x as String`
ternário `c ? a : b`	`if (c) a else b`
`int` versus `Integer`	só `Int`

> Os conceitos são os mesmos que vocês já conhecem, expressos de forma mais concisa.

O que muda — semântica

Aspecto	Java	Kotlin
Nulo	Qualquer referência pode ser nula	Está no **sistema de tipos**
Classes	Abertas por padrão	**Finais** por padrão
Exceções verificadas	`throws` obrigatório	**Não existem**
Mutabilidade	`var` é o padrão	`val` é o padrão
Coleções	Mutáveis por padrão	**Imutáveis** por padrão

> As três primeiras costumam ser as diferenças mais notadas por quem vem de Java. As três últimas são escolhas de projeto que favorecem código mais seguro.

Parte 3

Kotlin Multiplatform

O problema que a plataforma cria

O mesmo aplicativo, duas vezes:

```
Android              iOS
Kotlin               Swift
Jetpack Compose      SwiftUI
Android Studio       Xcode
Play Console         App Store Connect
```

São duas bases de código, duas equipes e dois cronogramas para o mesmo produto.

> É desse custo que surgem as soluções multiplataforma, cada uma com uma estratégia diferente para reduzi-lo.

O que é Kotlin Multiplatform

Um módulo Kotlin compilado para **várias plataformas**:

```
commonMain/          código que roda em todo lugar
   │
   ├── androidMain/    específico de Android    → bytecode JVM
   ├── iosMain/        específico de iOS        → binário nativo
   └── jvmMain/        desktop                  → bytecode JVM
```

Alvo	Como Kotlin vira executável
Android, desktop, servidor	**Kotlin/JVM** — bytecode, como Java
iOS, macOS, Linux, Windows	**Kotlin/Native** — binário, sem VM
Web	**Kotlin/JS** e **Kotlin/Wasm**

> A mesma linguagem, com três compiladores diferentes por trás.

Os componentes

Componente	O que faz
`commonMain`	O código compartilhado. Só usa a biblioteca padrão comum
Conjuntos por alvo	`androidMain`, `iosMain`, `jvmMain` — o que é específico
`expect` / `actual`	O mecanismo que liga os dois
Compose Multiplatform	Interface declarativa compartilhada (opcional)
Gradle	Orquestra tudo: alvos, dependências, artefatos
Bibliotecas KMP	Ktor, kotlinx.serialization, Room, DataStore, Koin

> `commonMain` não tem acesso às APIs de Android nem de iOS. Essa restrição é o que garante que o código comum realmente funcione em todos os alvos.

`expect` / `actual`

Quando algo só existe numa plataforma, o comum **declara** e cada alvo **implementa**:

```
// commonMain — declara o contrato
expect class ArmazenamentoSeguro() {
    fun salvar(chave: String, valor: String)
    fun ler(chave: String): String?
}

// androidMain — implementa com EncryptedSharedPreferences (específico do Android)
actual class ArmazenamentoSeguro actual constructor() {
    actual fun salvar(chave: String, valor: String) { /* ... */ }
    actual fun ler(chave: String): String? { /* ... */ }
}

// iosMain — implementa com Keychain (específico do iOS)
```

A estrutura é a mesma de interface e implementações que vocês já conhecem de orientação a objetos. A diferença é que a ligação é feita pelo compilador, por alvo, e não em tempo de execução.

> É o mecanismo que permite usar bibliotecas nativas sem abrir mão do código comum.

Compose Multiplatform

A interface, também em `commonMain`:

```
@Composable
fun CartaoObra(obra: Obra) {
    Column(modifier = Modifier.padding(16.dp)) {
        Text(obra.titulo, style = MaterialTheme.typography.titleMedium)
        Text("${obra.artista} · ${obra.ano}")
    }
}
```

É a **mesma API** do Jetpack Compose do Android, estendida a iOS, desktop e web.

> Quem escreve para Android com Jetpack Compose já está usando a mesma API do Compose Multiplatform.

Performance

	Como roda
Android	Mesmo bytecode de sempre. **Zero** camada extra
iOS	Kotlin/Native compila **ahead-of-time**, sem VM
Interface nativa	Componentes do sistema, não redesenhados
Compose Multiplatform	Renderiza via Skia sobre **Metal** no iOS

Não há uma ponte de serialização entre a lógica e a plataforma, como ocorre no React Native.

Quem usa em produção

- **Google** — Docs para Android, iOS e web, com lógica compartilhada em KMP
- **Netflix**, **McDonald's**, **Cash App**, **Duolingo**, **Forbes**, **Philips**
- **Sony** — aplicativo companheiro dos fones, com sensores e execução em segundo plano
- Bibliotecas do Jetpack já multiplataforma: **Room**, **DataStore**, **ViewModel**

Linha do tempo: Kotlin 1.0 em 2016 · oficial no Android em 2017 · Android *Kotlin-first* em 2019 · KMP estável em 2023 · Compose para iOS estável em 2025.

Por que KMP nesta disciplina

Motivo	Consequência prática
Kotlin é a linguagem nativa de Android	O que aprendem vale fora deste curso
Vocês já sabem Java	Aprendizado incremental, não reinício
A camada de domínio é Kotlin puro	Testes sem emulador, como em JUnit
A separação está no sistema de build	Dá para avaliar arquitetura no pipeline
Alvo desktop com recarga a quente	Ciclo rápido em Windows, Linux ou Mac

> Conforme o `docs/STACK.md`, a camada de domínio não importa Compose nem bibliotecas de infraestrutura. Nesta configuração, essa separação é verificada pelo próprio compilador.

Até semana que vem

- Quarta-feira (26/8) é dia de acompanhamento de projeto
- Esses dias são 100% online (pelo Meet) e não há conteúdo novo
- Oportunidade para tirar dúvidas e mostrar o que está sendo feito

Tarefas

- Formar o grupo do projeto, de 1 a 4 integrantes — até **11/09**
- Trazer uma ideia de produto, ainda que crua, no formato da visão da Parte 0
- Instalar o ambiente: **JDK 17+**, **Android Studio** com o plugin de KMP, e `kdoctor`
- Gerar o esqueleto em `kmp.jetbrains.com`

Recomendado

- **Kotlin Tour** · `kotlinlang.org/docs/kotlin-tour-welcome.html`
- Início rápido de KMP · `kotlinlang.org/docs/multiplatform/quickstart.html`
- Reserve cerca de 10 GB de disco e um tempo tranquilo para a instalação. O `kdoctor` indica o que estiver faltando.

Referências

A linguagem

- Kotlin Tour · `kotlinlang.org/docs/kotlin-tour-welcome.html`
- Kotlin Docs · `kotlinlang.org/docs/home.html`
- Comparação com Java · `kotlinlang.org/docs/comparison-to-java.html`

Multiplataforma

- Kotlin Multiplatform · `kotlinlang.org/docs/multiplatform/`
- Compose Multiplatform · `kotlinlang.org/compose-multiplatform/`
- Assistente de projeto · `kmp.jetbrains.com` · Catálogo · `klibs.io`

Android

- Android Basics with Compose (Google) · `developer.android.com/courses`

Disciplina

- `github.com/fmarquesfilho/sistemas-moveis-2026-2`